

РАЗРАБОТКА СИСТЕМЫ ДЛЯ ПРОКСИРОВАНИЯ И МОНИТОРИНГА ИМИТАЦИОННЫХ СЕРВИСОВ В СРЕДЕ НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ

Студент:
Медведев Клим Николаевич,
группа 221-329.
Научный руководитель:
Кружалов Алексей Сергеевич,
старший преподаватель.

При нагрузочном тестировании сложных систем требуются десятки имитационных сервисов (заглушек). Ручное управление ими — трудоёмко и ненадёжно.

Проблемы существующих решений:

- WireMock, MockServer, Mountebank не управляют инфраструктурой JAR/WAR-процессов
- Apache Tomcat избыточен и не поддерживает динамический Rate Limiting
- Все зарубежные решения косвенно совместимы с ОС Astra Linux

Вывод: ни одно из существующих решений не закрывает задачу централизованного управления распределённой инфраструктурой заглушек в условиях импортозамещения

Цель и задачи работы

Цель: спроектировать и разработать автоматизированную информационную систему для централизованного управления имитационными сервисами в среде нагрузочного тестирования и мониторинга их состояния.

Задачи:

1. Провести анализ предметной области и существующих подходов
2. Спроектировать архитектуру программного комплекса
3. Обосновать выбор технологического стека
4. Разработать алгоритмы проксирования, Rate Limiting и API управления
5. Реализовать систему хранения конфигураций и контроля состояний
6. Провести тестирование в среде ОС Astra Linux

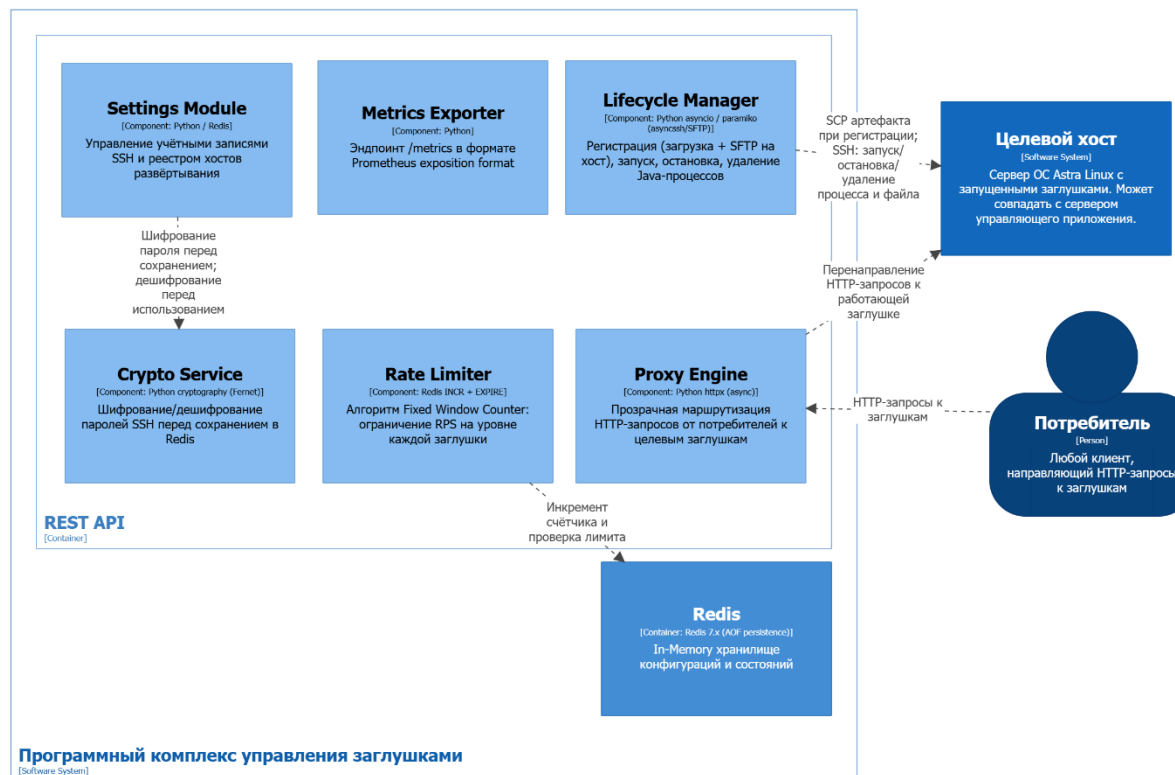
Сравнительный анализ существующих решений

Критерий	WireMock	MockServer	Apache Tomcat	Разраб. система
Управление JAR/WAR	Нет	Нет	Да	Да
Централизованная оркестрация	Нет	Нет	Частично	Да
Динамический Rate Limiting	Нет	Нет	Нет	Да
Потребление ресурсов	Высокое (JVM)	Высокое (JVM)	Высокое	Низкое
Совместимость с Astra Linux	Частично	Частично	Частично	Да

Архитектура и технологический стек

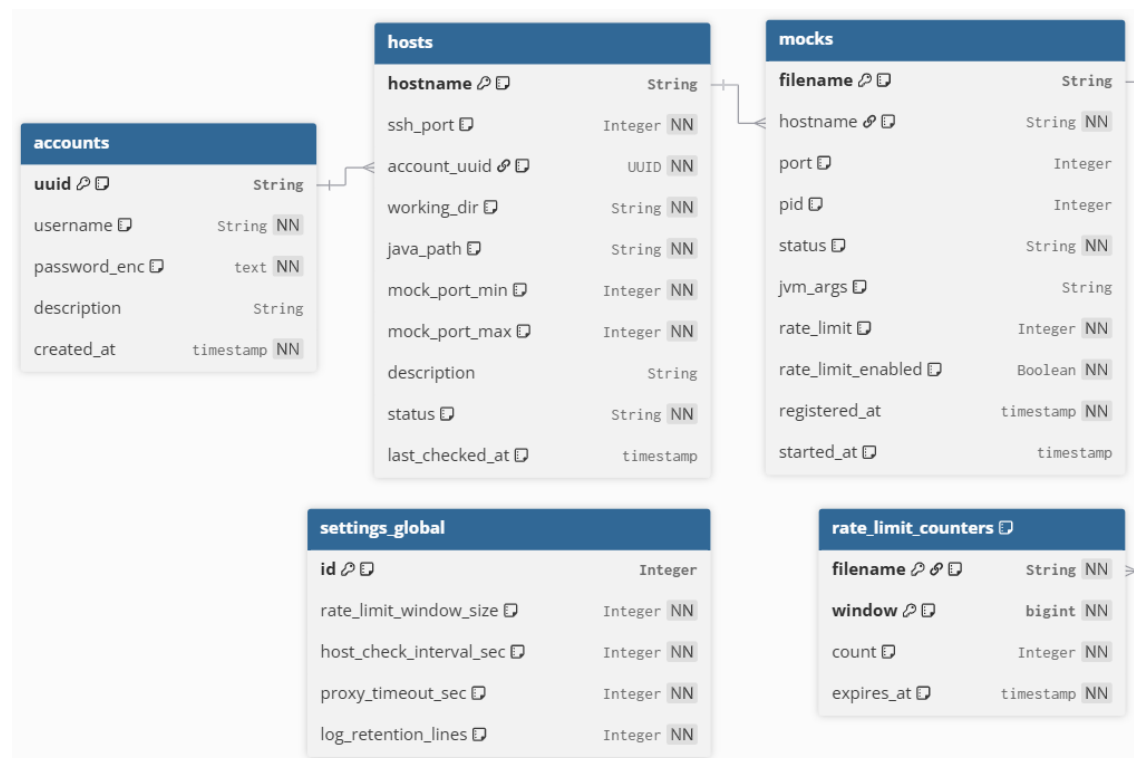
Выбрана монолитная архитектура с выраженной внутренней модульностью. Управление удалёнными хостами — через SSH/SFTP без установки агентов.

Технологический стек: Python + asyncio / FastAPI + Uvicorn / Redis AOF / Astra Linux



Структуры данных

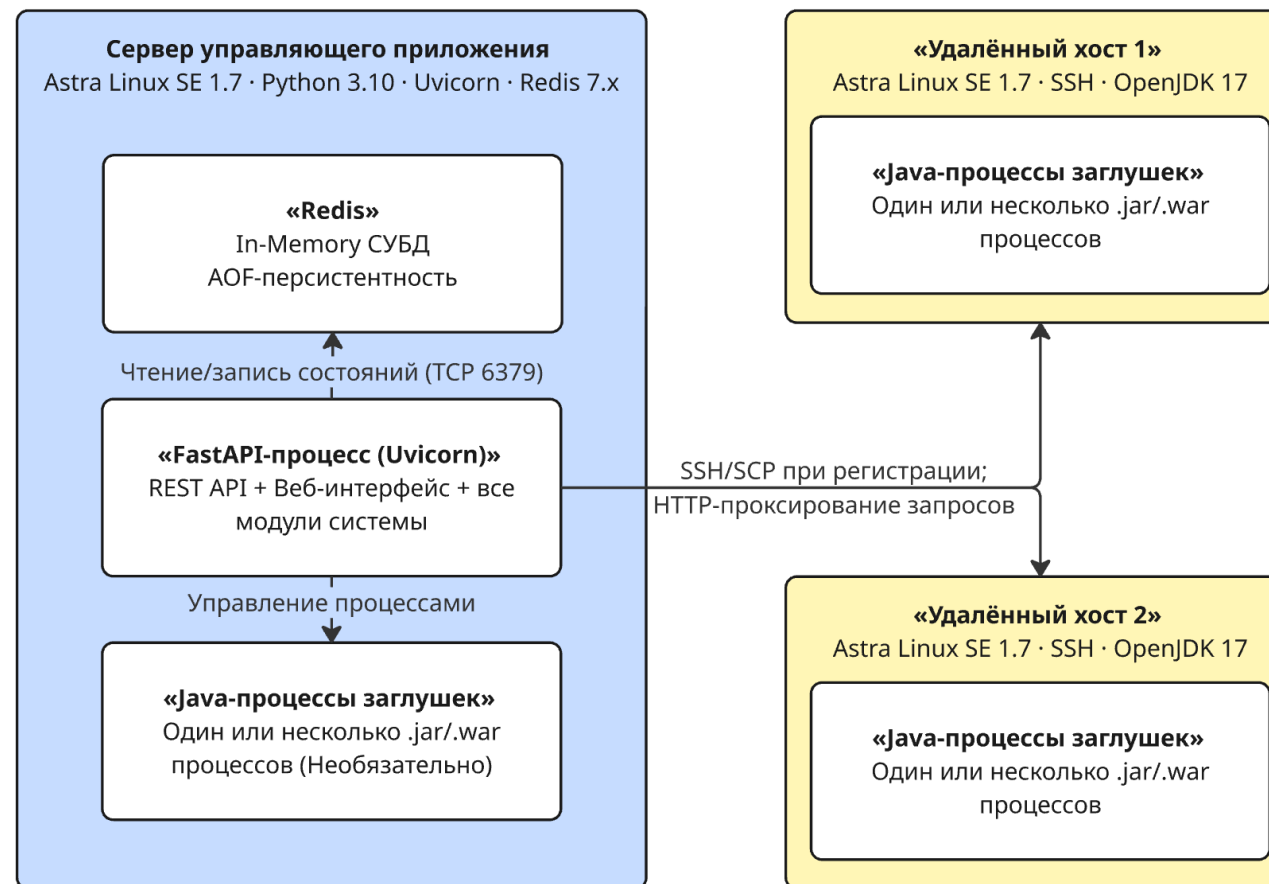
Для каждого объекта системы используется Hash-запись — позволяет читать конфигурацию целиком за одну операцию HGETALL и обновлять отдельные поля через HSET без перезаписи всей записи. Бинарный файл заглушки хранится только на целевом хосте — передаётся по SFTP при регистрации и немедленно удаляется с управляющего сервера.



Топология развёртывания

Система поддерживает два режима без изменения кодовой базы:

- **Cluster Mode:** один управляющий узел координирует несколько удалённых хостов через SSH/SFTP
- **Standalone Mode:** все компоненты на одном сервере
На целевых хостах не требуется установка агентов — только SSH-сервер и OpenJDK 17.



Проксирование и Rate Limiting

Прокси-модуль прозрачно маршрутизирует запросы: сохраняет метод, заголовки и тело.
При превышении лимита возвращает HTTP 429, не нагружая заглушку.

```
def check_rate_limit(filename: str, limit: int, window_size: int = 1) -> bool:
    # Шаг 1: вычислить метку текущего окна
    window = floor(current_unix_timestamp() / window_size)
    key = f"rate:{filename}:{window}"
    # Шаг 2: атомарный инкремент счётчика
    count = INCR(key)
    # Шаг 3: установить TTL при создании нового окна
    if count == 1: EXPIRE(key, window_size * 2) # TTL = 2 окна (запас на сдвиг времени)
    # Шаг 4: проверить лимит
    if count > limit: return False # запрос отклонён -> HTTP 429
    return True # запрос разрешён
```


Запуск на удалённом хосте выполняется через SSH без установки агентов — командой `nohup`, PID возвращается в `stdout` и сохраняется в Redis.

Остановка реализует двухэтапный протокол: сначала `SIGTERM`, затем при отсутствии реакции в течение 5 секунд — `SIGKILL`. Оба этапа выполняются асинхронно, не блокируя обработку других запросов. При перезапуске системы `restore_state` проверяет фактический PID каждой заглушки и автоматически перезапускает завершившиеся процессы с сохранёнными JVM-аргументами. `HealthChecker` обнаруживает сбой за ≤ 30 с.

Реализован на Server-Side Rendering (FastAPI + Jinja2) без внешних зависимостей.
Статус каждой заглушки кодируется цветом: ● RUNNING / ● STOPPED / ● ERROR.
Таблица обновляется автоматически каждые 5 секунд.

MockControl

Dashboard

Настройки

Журналы

Dashboard

Заглушки

ВСЕГО ЗАГЛУШЕК
2

ЗАПУЩЕНО
2

ОШИБОК
0

ХОСТОВ ДОСТУПНО
2

+ Зарегистрировать заглушку

ИМЯ ФАЙЛА	ХОСТ	СТАТУС	ПОРТ	RATE LIMITER	ДЕЙСТВИЯ
uuid-stub-jdk8-2.1.war	192.168.1.77	RUNNING	8100	☒	Стоп Удалить
uuid-stub-jdk8-fixed-local.war	192.168.1.76	RUNNING	8100	☐	Стоп Удалить

Результаты функционального тестирования

Сценарий	Успешность	Результат
Перезапуск при работающих процессах		Статусы RUNNING сохранены
Перезапуск при завершённых процессах		Автоматический перезапуск заглушек
Восстановление после аварии Redis		Данные восстановлены из AOF-журнала
Обнаружение сбоя планировщиком HealthChecker		Статус ERROR за ≤ 30 с
Автовозврат из ERROR в RUNNING		Статус RUNNING за ≤ 30 с

Нагрузочное тестирование

Результаты нагрузочного тестирования прокси-модуля
Инструмент: k6. Наблюдение через эндпоинт /metrics (Prometheus).

Сценарий	Показатель	Результат
Базовая нагрузка (10 RPS)	proxy_total_ms	3,90 мс
Нарастающая нагрузка (500 RPS)	proxy_total_ms	3,01 мс — линейный рост
Нарастающая нагрузка (1000 RPS)	proxy_total_ms	10,52 мс — без отказов
Rate Limiting (лимит 250, нагрузка 500 RPS)	Отклонение от лимита	0,13%

Все 6 задач выполнены в полном объёме.

Разработан программный комплекс, не имеющий прямых аналогов среди существующих решений.

Ключевые характеристики:

- Задержка прокси: $\leq 3,01$ мс при нагрузке до 500 RPS
- Точность Rate Limiting: отклонение 0,13% от теоретического лимита
- Время обнаружения сбоя: ≤ 30 с
- Научная новизна: агентно-независимое управление распределённой средой заглушек с индивидуальным Rate Limiting для каждого сервиса без перезапуска
- Направления развития: динамическая маршрутизация по содержимому запроса, ролевая модель доступа, поддержка не-JVM артефактов

**РАЗРАБОТКА СИСТЕМЫ ДЛЯ ПРОКСИРОВАНИЯ И
МОНИТОРИНГА ИМИТАЦИОННЫХ СЕРВИСОВ В СРЕДЕ
НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ**

Студент:
Медведев Клим Николаевич,
группа 221-329.
Научный руководитель:
Кружалов Алексей Сергеевич,
старший преподаватель.